

DevFlow

边界驱动的多 Agent 研发闭环

以 AgentTeams 为协同基点 | 职责清晰、交接可验、工具默认拒绝

复杂研发问题，失败往往发生在协作边界

场景与价值

同一 Issue 在分类、定位、修改、测试、审查之间流转。身份、证据与权限一旦混在聊天里，结果就难复验；DevFlow 把每次协作收敛为可验证合同。

6

专职角色
职责不漂移

7

可复用 Skill
输入输出可验

24 项

路由 / 边界基准
不是补丁解决率

六角色不是角色扮演：每个身份只有一个可追责出口

TeamLeader

拆解 DAG、跟踪状态、冲突仲裁、失败重排；不写代码、不跑测试。

Triage

issue-classifier → ClassifiedIssue；不读写仓库。

Locator

固定 revision 只读定位；不得修改、执行或扩大路径范围。

Coder

仅产最小补丁候选；不测试、不推送、不合并。

Tester

一次性副本验证；只运行服务器预注册 CI 动作。

Reviewer

独立审查与经验提炼；不合并、不部署、无 GitHub 写能力。

HumanReviewer

系统外部授权主体；只批准 T4/T5，不计入六个 Agent。

冲突优先级

安全 > 人工策略 > 测试证据 > 审查 > Coder 自检。

身份可执行，交接可校验 房间消息只做协作，不做授权

01 | 我是谁

进程身份 + Worker 角色 + Skill owner。
提示词里的“我是 Leader”不能改变进程身份。

02 | 我拿到什么

HandoffEnvelope 绑定 producer /
consumer / task / Skill / payload type /
SHA-256 / trace / idempotency。

03 | 我能做什么

身份 $\cap$ Skill $\cap$ MCP $\cap$ 参数 $\cap$ 上下文 $\cap$
人工批准。任一未知、错配或过期，默认
拒绝。

AgentTeams 五项硬映射：从框架对象到协作语义

ROLE / TASK

CONTEXT / RUN

STATE / TRACK

角色编排与任务拆解

Manager 建 Team；Leader 拆 DAG 并定向 assignment；Worker 只 ACK 自己任务。

上下文传递与协同执行

Team Room 共享状态；Worker Room 保持最小上下文；Handoff 验后只调用所属 Skill。

状态追踪

taskflow 跟踪 ACK/结果/验收；readback 完成，失败 retry，高风险 PAUSED。

GitHub 只读能力：请求必须穿过五道边界

任务上下文	短期能力	Higress	Scope Broker	GitHub API
Leader: 签发固定 repo / revision / path				
	Locator: 仅 github-evidence, 可读不可写			
	Higress: Consumer=Locator; Wasm FAIL_CLOSE			
		Broker: 复核能力签名、Agent + Skill 与精确 scope		
			回执: Git object + content digest, 可独立重算	
默认拒绝: 错路径 403 Reviewer 403 Locator 直连 Broker 被阻断				

七个 Skill 的工程标准：可调用、可拒绝、可验证、可复用

issue-classifier

Triage | 分级、去重、优先级 →
ClassifiedIssue

code-root-cause

Locator | 固定 revision 根因与影响
面 → LocatedContext

github-evidence

Locator | 固定 repo / revision /
path 的只读证据

patch-generator

Coder | 最小候选修复 →
PatchCandidate

test-runner

Tester | 隔离副本、基线与回归 →
TestEvidence

pr-reviewer

Reviewer | 正确性、安全性与是否
可进入人工门

experience-distiller

Reviewer | 脱敏复盘 → 可复用经验

统一质量门

输入输出、触发拒绝、依赖失败、
角色归属、安全验证、版本；7/7 静态
100/100。

真实环境已验证的不是“会说”，而是状态与拒绝

现场证据 + 本地发布门

2026-07-27 AgentTeams v1.2.0-beta.1 现场；2026-07-28 DevFlow v1.3.0 本地门。数字分别对应项目状态、边界负例与本地质量。

2 节点

Triage → Reviewer
completed; pending=false

3 层

错路径 / 错身份 / 直连
分别被拒绝或阻断

84.40%

846 passed / 17 skipped
7/7 Skill 静态 100/100

证据矩阵：把“现场”“测试”“待验证”明确分层

证据纪律

每条能力只使用与其成熟度匹配的表述；真实集群、本地确定性演示与边界基准不得拼接成未发生的完整修复。

能力	证据类型	正例	负例 / 恢复	对外口径
Bot 通信	现场	Leader→Triage→Leader	丢消息/流式回归已修复	真实消息链
项目状态	现场	T1/T2 completed	Matrix 通知自动重试	仅两节点
进程身份	现场	Leader 控制项目	冒充 Leader 被拒	非强 sandbox
GitHub scope	现场	固定 README 读取	换路径 403	单 repo/path
Higress 身份	现场	Locator 放行	Reviewer 403	只读入口
网络路径	现场	仅经网关成功	直连 Broker 阻断	网关专属
Skill 质量	发布门	7 项静态 100/100	验证器可 python -S	非 7 项行为分
24 项基准	固定 revision	24/24 精确决策	6/6 安全决策	非修复率

官方评分不是堆工具：五项权重围绕闭环证据

75% | 价值 + 协同 + Skill

场景 25%：真实问题与可复制价值。

协同 25%：身份、状态、异常与人工门。

Skill 25%：七项资产与复用边界。

20% | 工程、安全、审计

可运行材料；日志、Trace、Metrics。

MCP、RAG 与观测链可验证。

密钥、审批、回滚、降级、审计不缺位。

5% | 开放 / 开源

Apache-2.0、接口契约、README、复现实例与贡献说明。公开仓库、最终 tag 与提交包一致性仍需提交人终检。

距离“夺冠作品”还差三条现场闭环

P0 · 六阶段修复

同一项目跑完整链

Triage→定位→修复→测试→审查→经验沉淀；保留原始失败、补丁、绿测与canonical 未改证据。

P0 · 失败回传

红测必须回 Coder

Tester 生成脱敏有界证据；Leader 校验后转 Coder，不得改写红测结论。

P0 · T4 人工门

先拒绝，再签名恢复

paused→未批准恢复失败→精确摘要签名→resume+审计。完成后再录正式视频并冻结最终覆盖率。

DEVFLOW •
PRELIM

把协作变成证据 把权限变成边界

已验证 | 六角色部署 / 两节点闭环

已验证 | GitHub 三层负例 / 摘要回执

待现场 | 六阶段 / 红测闭环 / T4 真人恢复